

OSTIM TECHNICAL UNIVERSITY
ANKARA

T.C.
OSTIM TECHNICAL UNIVERSITY
Faculty of Engineering

Department of Electrical and Electronics Engineering

Smart Police Patrol

210202887 ABDALLA ALI

220202927 ABDULLAH ALHADEETHI

210202963 MOHAMED ABDULLAHI

200202948 ELHASSAN ABDELMOHSEN

Prof.Dr. UFUK PARALI

May 2026
ANKARA



OSTIM TECHNICAL UNIVERSITY
ANKARA

T.C.
OSTIM TECHNICAL UNIVERSITY
Faculty of Engineering

Department of Electrical and Electronics Engineering

Smart Police Patrol

210202887 ABDALLA ALI

220202927 ABDULLAH ALHADEETHI

210202963 MOHAMED ABDULLAHI

200202948 ELHASSAN ABDELMOHSEN

Prof.Dr. UFUK PARALI

May 2026
ANKARA

GRADUATION PROJECT APPROVAL FORM

The undergraduate graduation project titled “Smart Police Patrol System”, prepared by ABDALLA ALI – ABDULLAH ALHADEETHI - MOHAMED ABDULLAHI - ELHASSAN ABDELMOHSEN under the supervision of Doç.Dr. UFUK PARALI was examined and accepted as an Undergraduate Graduation Project in terms of its scope and quality.

Supervisor	:	Prof. Dr. Ufuk PARALI
Jury member 1	:	Prof. Dr. Şenol GÜLGÖNÜL
Jury member 2	:	Dr. Öğr. Üyesi Arda KILIÇ
Head of the Department	:	Prof. Dr. İSMAİL HAKKI ALTAŞ

FOREWORD

This graduation project presents the design and development of a Smart Police Patrol System based on edge computing and artificial intelligence technologies. The project was carried out within the Department of Electrical and Electronics Engineering at OSTIM Technical University as part of the requirements for the undergraduate graduation project. Throughout this study, various modern technologies such as YOLOv8 object detection, Optical Character Recognition (OCR), and embedded systems were integrated to develop a low-cost and real-time vehicle monitoring solution capable of operating without cloud dependency. The project provided valuable practical experience in computer vision, software development, and system integration.

We would like to express our sincere gratitude to our supervisor, Doç. Dr. UFUK PARALI, for his continuous guidance, valuable suggestions, and support throughout all stages of this project. His feedback and encouragement played a major role in improving both the technical and academic quality of this work.

We would also like to thank the faculty members of the Department of Electrical and Electronics Engineering for their support and for providing a suitable academic environment during our studies.

Finally, we are deeply grateful to our families and friends for their patience, encouragement, and continuous support throughout our educational journey.

May 2026

ABDALLA ALI
ABDULLAH ALHADEETHI
MOHAMED ABDULLAHI
ELHASSAN ABDELMOHSEN

CONTENTS

Graduation Project Approval Form i
Foreword ii
Contents iii
Abstract iv
Symbols and Abbreviations v
The List Of Figures vi
The List Of Tables vii

1. INTRODUCTION 1
1.1 General Information 1
1.2 Literature Review 1
1.3 Originality (Novelty) 2
1.4 Methodology 3
1.5 Impact 3
1.6 Standards 4
1.7 Work Schedule 4
1.8 Risk Analysis 6
1.9 Organization of Work Packages 7

2. THEORETICAL BACKGROUND 8
2.1 General Information 8
2.2 Intelligent Systems and Computer Vision 8
2.3 Microprocessors and Edge Computing Hardware 9
2.4 Image Processing Techniques 10
2.5 Human-Machine Interaction (HMI) 10

3. DESIGN 12
3.1 General Information 12
3.2 Dimensioning 13
3.3 System Components and Their Selections 14
3.4 Applied Methods 15
3.5 Software Used 15
3.6 Component List and Economic Analysis 18
3.7 Legal Aspects 18

4. SIMULATION STUDIES 19
4.1 General Information 19
4.2 Simulation Software 20
4.3 System Modeling 21
4.4 Simulation 22

5. EXPERIMENTAL WORKS 24

6. RESULTS 25
6.1 General Information 25
6.2 Simulation Results 26
6.3 Simulation-Based Testing Results 27

7. CONCLUSIONS 30
7.1 Comments and Evaluation 30
7.2 Problem Solving and Facilitation 30
7.3 Future Work 30
7.4 Budget and Design Comparison 30

8. REFERENCES 32

ABSTRACT

This project presents the design and simulation of a Smart Police Patrol System based on edge computing, YOLOv8 object detection, and OCR. The system detects vehicle license plates from a live video feed, recognizes plate characters, and compares them with a local database to generate real-time alerts. The proposed architecture reduces cloud dependency and supports offline operation in patrol vehicles. Simulation-based testing results show that the system can detect and classify plates with high accuracy under different controlled conditions.

SYMBOLS AND ABBREVIATIONS

ABBREVIATIONS

AI : Artificial Intelligence

ALU : Arithmetic Logic Unit

ANPR : Automatic Number Plate Recognition

BGR : Blue, Green, Red

COTS : Commercial Off-The-Shelf

CPU : Central Processing Unit

CSI : Camera Serial Interface

DSI : Display Serial Interface

FPS : Frames Per Second

GPIO : General-Purpose Input/Output

GUI : Graphical User Interface

HMI : Human-Machine Interaction

IoT : Internet of Things

IoU : Intersection over Union

KVKK : Kişisel Verilerin Korunması Kanunu (Law on the Protection of Personal Data)

LPR : License Plate Recognition

mAP : mean Average Precision

MIPI : Mobile Industry Processor Interface

NMS : Non-Maximum Suppression

OCR : Optical Character Recognition

OS : Operating System

OTU : OSTİM Technical University

RGB : Red, Green, Blue

SBC : Single Board Computer

WP : Work Package

YOLO : You Only Look Once

SYMBOLS

$\boldsymbol{S} \times \boldsymbol{S}$: Grid dimensions for YOLO image division

$\boldsymbol{Pr}(\boldsymbol{Object})$: Probability that a target object exists within a grid cell

$\boldsymbol{P}_{\{x,y\}}$: Pixel intensity at coordinates (x, y)

$\boldsymbol{I}$: Input video frame

$\boldsymbol{B}$: Localized bounding box

$\boldsymbol{c}$: Confidence score of the detection

$\boldsymbol{S}$: Extracted alphanumeric text string

$\boldsymbol{D}$: Local database dictionary

$\boldsymbol{O}$: Operational output status (Clear, Warning, Critical)

THE LIST OF FIGURES

Figure 2.1. YOLO Object Detection Theoretical Framework 8
Figure 3.1. Hardware Connection Diagram 13
Figure 4.1. System Model Architecture 22
Figure 4.2. Software Flowchart 24
Figure 6.1. YOLOv8 Training and Validation Metrics over 50 Epochs 26
Figure 6.2. Live demonstration of the system detecting a Turkish plate with "Unpaid Fines" (Yellow bounding box) 28
Figure 6.3. Live demonstration of the system processing a Russian plate in optimal conditions 28
Figure 6.4. Live demonstration of the system detecting a Saudi plate and triggering a "Stolen Vehicle" alert (Red bounding box) 29

THE LIST OF TABLES

Table 1.1. Work-Time Table 5

Table 1.2. Risk Analysis and Plan B 6

Table 3.1. Component List 17

Table 6.1. Experimental Testing Results under Real-World Conditions 29

1. INTRODUCTION

1.1. General Information

Law enforcement agencies today rely heavily on technology to regulate traffic and maintain public safety. Currently, the standard practice involves using cloud-connected Automatic Number Plate Recognition (ANPR) cameras mounted on highway poles or premium police vehicles. We chose to tackle this specific topic because these existing systems often struggle with significant delay issues in areas with poor network coverage, and they are incredibly expensive to deploy at scale.

The Smart Police Patrol System is an embedded hardware and software solution built to solve this exact problem. It acts as an offline assistant that monitors the streets directly from the patrol car's dashboard. By combining a low-cost Raspberry Pi with the YOLO object detection algorithm, the system processes video frames entirely locally. The main innovation here is shifting the heavy computational load away from distant servers and bringing it right into the vehicle. This structural change provides officers with instant, offline visual alerts for stolen vehicles or expired insurance, actively reducing their cognitive load and letting them focus on safe driving.

1.2. Literature Review

The integration of computer vision in intelligent transportation and law enforcement has been widely explored, with a recent shift towards edge computing and lightweight deep learning models. Early systems of License Plate Recognition (LPR) primarily depended on traditional image processing techniques such as Sobel edge detection and Haar-like features, which required heavy computational resources and struggled with real-time performance [1].

To address these limitations, recent engineering studies have heavily focused on the YOLO (You Only Look Once) architecture for real-time object detection. For instance, an optimized YOLO-based system was proposed for efficient LPR, demonstrating that Edge AI deployment on devices like the Raspberry Pi enables real-time inference with minimal latency while maintaining over 95% mean Average Precision (mAP) [2]. Similarly, researchers in Turkey developed an automatic license plate recognition and visualization system utilizing the YOLOv8 algorithm, achieving up to 99% accuracy on Turkish (TR) plates without relying on external cloud processing [3]. Furthermore, studies have proven that dynamic license plate recognition using YOLO performs exceptionally well on data collected directly from a video stream using a Raspberry Pi [4].

Beyond plate recognition, edge computing is actively being used to monitor operational safety and driver behavior. Recent literature highlights the development of low-cost driver monitoring systems utilizing a Raspberry Pi and deep learning to identify states such as distracted driving and smartphone usage with high accuracy [5]. Additionally, deep learning frameworks and multi-view imaging are currently being employed to recognize distracted driving behaviors in real-time by analyzing visual distractions inside the vehicle [6].

Summary and Project Positioning: While the existing literature provides a strong foundation in edge computing and YOLO-based detection [1]–[6], there is a noticeable gap in combining multi-hazard detection (e.g., wanted plates and distracted driving) into a single, offline, low-cost embedded dashboard specifically designed for moving police vehicles. The Smart Police Patrol System addresses this gap by creating a unified, real-time alert interface that operates entirely on edge hardware.

1.3. Originality (Novelty)

While ANPR technology itself is a well-known concept, this project introduces specific structural and design differences compared to standard commercial systems. Traditional setups are largely stationary and rely on sending heavy video data to cloud servers for processing. When applied to moving police units, this traditional structure is prone to unacceptable lag due to dropping cellular signals.

The originality of our study lies in its hardware-software co-design tailored specifically for edge-AI. Instead of treating the dashboard camera as just a recording device, we restructured the entire workflow to process data directly on an affordable embedded board inside the car. This design difference completely removes internet dependency, ensuring real-time performance regardless of the patrol location. Additionally, rather than building a rigid license plate scanner, we engineered the system as a flexible computer vision platform. As our simulation testing shows, this specific architectural choice means the core AI model can be easily adapted to detect secondary traffic hazards like distracted drivers, making it a uniquely versatile and budget-friendly tool for local security forces.

1.4. Methodology

The methodology for the Smart Police Patrol System is structured as a complete engineering workflow, systematically moving through five main stages. During the idea generation phase, we analyzed real-world traffic scenarios to identify the limitations of current police dashcams, focusing specifically on network latency and officer distraction. For the literature review, we examined recent academic papers regarding edge computing and YOLO-based object detection to establish a solid, evidence-based technical foundation.

In the design stage, we adopted a hardware-software co-design approach. Instead of a cloud-dependent architecture, we structured the system around a local edge device (such as a Raspberry Pi) running a custom computer vision pipeline. The

simulation phase, which forms the core of Graduation Project I, involves testing this AI logic using recorded video feeds and printed test cases to validate the detection accuracy and the dashboard GUI. Finally, the realization stage planned for Graduation Project II will involve the physical assembly of these hardware components.

1.5. Impact

The completion of this project will deliver multi-dimensional benefits. Locally, it solves the immediate problem of delayed threat detection for patrol units by operating entirely offline. On a national level, adopting such low-cost edge-AI systems can modernize public safety infrastructure without requiring massive governmental budgets.

Economically, this standalone approach reduces dependency on expensive, imported surveillance hardware. This supports domestic technological production and offers cost-saving alternatives for municipalities. Regarding employment, it creates a shift toward data-assisted policing, where officers are supported rather than replaced, allowing human resources to be allocated to more critical physical tasks.

The project also carries indirect health and environmental benefits. By automating continuous visual monitoring, it drastically lowers the cognitive burden and stress on police drivers, contributing to safer road navigation (occupational health). Environmentally, faster detection of flagged vehicles can reduce unnecessary pursuits and patrol idling times. Socially, processing video data locally on the edge inherently protects civilian privacy, preventing sensitive footage from being transmitted over vulnerable public networks.

Given its robust integration of hardware and deep learning, this study has strong potential for publication. The technical findings regarding edge-AI performance in moving vehicles could be published in undergraduate engineering symposiums, the TÜBİTAK Turkish Journal of Electrical Engineering & Computer Sciences, or IEEE conference proceedings focusing on intelligent transportation systems.

1.6. Standards

To ensure the safety, reliability, and technical quality of the Smart Police Patrol System prototype, the development and hardware assembly processes comply with several established international and national standards. Since Graduation Project II involves physically wiring and assembling an edge computing board (Raspberry Pi) and a camera module for a tabletop demonstration, we prioritized safety and software integrity. The selected TSE, IEC, and IEEE standards are listed in the table below:

Standard Number	Standard Name	Project Relevance
IEC 62368-1:2023	Audio/video, information and communication technology equipment - Part 1: Safety requirements	Applied to ensure the safe electrical integration of the Raspberry Pi, camera module, and display monitor during the hardware assembly phase.
IEC 61508-3:2010	Functional safety of electrical/ electronic/programmable electronic safety-related systems - Part 3: Software requirements	Guides the software development lifecycle for our YOLO-based computer vision script, ensuring the Python code runs reliably without unexpected crashes.
TS EN ISO/IEC 27001	Information Security Management System	Serves as the guiding framework for handling the system's mock security database, emphasizing the importance of data privacy for vehicle plates.
IEC 60529:1989	Degrees of protection provided by enclosures (IP Code)	Used as a reference standard when selecting or designing an appropriate protective case for the sensitive edge computing hardware.
IEEE 802.11-2020	IEEE Standard for Information Technology - Telecommunications and Information Exchange Between Systems	Ensures standard compliance for the Wi-Fi module of the Raspberry Pi, which is used to remotely update the local alert database.

1.7. Work Schedule

This section outlines the work plan organized into sequential Work Packages (WP) to ensure the systematic progression of the Smart Police Patrol System. The workload is distributed among four team members (Abdalla Ali, Mohamed Abdullahi, ELhassan Ali, and Abdullah Natheer). Following project management guidelines, each student acts as the leader (L) for at least one package to ensure a balanced distribution of responsibility.

Table 1.1. Work-Time Table

WP No	Name and Purpose of the Work Package	Working Team	Time interval	Contribution to the Project
1	Creating the project idea, literature research and revealing the original idea	Abdalla Ali (L)	Feb 2026	The project idea is created and originality is supported by literature research. The formation of the original idea also clarifies what will be done in the next steps.
2	Determination of the methods to be applied and related theoretical studies. Proceeding to the design phase	Mohamed Abdullahi (L) Abdullah alhadeethi	Feb 2026	The methods to be applied are supported by theoretical explanations. Continuation of the design depends on the adequacy of theoretical knowledge.
3	Completion of design calculations and drawings, financial analysis and budgeting, and investigation of legal responsibilities	ELhassan abdelmohsen (L) Mohamed Abdullahi	Mar 2026	Design calculations and technical drawings of the project are made. Connection diagrams are created according to the dimensions of the components such as table, case, box etc. Connection diagrams are the working indicator of the prototype.
4	Creating the simulation model, performing simulations, evaluating the results and writing the Graduation Project book	Abdalla Ali (L) ELhassan abdelmohsen Abdullah alhadeethi Mohamed Abdullahi	Apr 2026	The simulation model of the designed system is created and simulated with package programs or software to be developed. Simulation is important as it will provide prior knowledge of whether the prototype will work or not.
5	Final exams and Graduation Project presentations,	Abdullah alhadeethi (L) ELhassan abdelmohsen Abdullah Ali Mohamed Abdullahi	May 2026	The prepared Engineering Design is evaluated by the jury during the presentations. Since Engineering Design is a prerequisite for the Graduation Project, it is important to successfully complete the presentations.

1.8. Risk Analysis

Table 1.2. Risk analysis

WP No	Contribution to the Project	Risk Analysis
1	The project idea is defined by focusing on the need for a low-cost smart patrol system that can detect vehicle license plates in real time. Literature research supports the originality of using edge computing for police patrol applications.	Similar license plate recognition systems already exist. As a Plan B, the project originality will be emphasized by focusing on offline operation, low-cost hardware, and real-time use inside patrol vehicles.
2	The theoretical background of YOLOv8, OCR, image processing, and edge computing is studied. These topics provide the technical foundation required to design and simulate the Smart Police Patrol System.	If the theoretical knowledge is not sufficient, additional academic sources and technical documentation will be reviewed. The selected methods can also be simplified according to the project scope.
3	The system design is prepared by selecting the camera, Raspberry Pi, display, power supply, and required connections. The design stage shows how the hardware and software parts work together.	The selected components may not provide the required processing speed. As a Plan B, a lighter YOLOv8 model, lower image resolution, or laptop-based testing can be used.
4	The simulation model is created using Python, YOLOv8, OCR, and a local database. This stage helps verify the system logic before moving to complete hardware implementation.	The simulation may not fully represent real road conditions such as motion blur, low light, and different viewing angles. Additional test images and videos will be used to improve reliability.
5	The engineering design report and presentation are prepared to explain the project idea, methodology, design steps, simulation results, and expected system performance to the jury.	The report may contain formatting, spelling, or consistency errors. As a Plan B, the project book will be reviewed carefully before submission and corrected according to the required format.
6	The required materials are selected and prepared for the prototype, including the Raspberry Pi, camera module, touchscreen display, cooling unit, storage, cables, and power supply.	Some components may not be available on time or may be expensive. As a Plan B, compatible alternative components can be ordered online or replaced with equivalent hardware.
7	The hardware connections are completed between the camera, Raspberry Pi, display, and power supply. Correct wiring is necessary to ensure stable system operation during testing.	Incorrect connections or unstable power may cause the system to fail. As a Plan B, each component will be tested separately before full system integration.
8	The prototype assembly is completed and prepared for functional testing. This stage allows the team to evaluate detection accuracy, OCR performance, and alert generation in practice.	The prototype may face overheating, camera instability, or display connection problems. As a Plan B, active cooling, secure mounting, and backup cables will be prepared.
9	The final project book is completed by explaining the system design, software implementation, simulation results, experimental observations, legal aspects, and overall project evaluation.	Missing sections, weak explanations, or inconsistent results may reduce the quality of the report. As a Plan B, all chapters, tables, figures, and references will be checked before submission.
10	The working prototype is demonstrated during the Graduation Project Exhibition. The system shows license plate detection, OCR reading, database checking, and visual alert generation.	The prototype may fail during the exhibition due to software, lighting, or hardware problems. As a Plan B, a backup video, screenshots, printed plates, and backup code will be prepared.

1.9. Organization of Work packages

The work packages are organized according to the two stages of the graduation project. The first five work packages are related to the first stage, which includes defining the project idea, reviewing the literature, studying theoretical background, completing the system design, preparing the simulation model, and presenting the engineering design. The last five work packages are related to the second stage, which includes material procurement, hardware connection, prototype assembly, system testing, final report completion, and project exhibition.

The responsibilities are distributed among the project members to ensure balanced participation and regular progress. Abdalla Ali is responsible for leading the project idea, literature review, simulation model, and software development tasks. Mohamed Abdullahi is responsible for the theoretical background, method selection, hardware connection, and final demonstration. Elhassan Abdelmohsen is responsible for system design, component selection, economic analysis, legal aspects, and material procurement. Abdullah Alhadeethi is responsible for the report preparation, presentation, prototype assembly, and testing process.

Each leader follows the progress of the assigned work package and coordinates with the other members to complete the required tasks on time. This organization helps the team manage the project systematically and reduces the possibility of delays during both stages of the graduation project.

2. THEORETICAL BACKGROUND

2.1. General Information

The theoretical foundation of the Smart Police Patrol System lies at the intersection of embedded systems and computer vision. In modern traffic enforcement, relying solely on human observation is no longer sufficient or safe. The general framework of this study centers on translating raw visual data into actionable intelligence directly at the source, a paradigm known as edge computing.

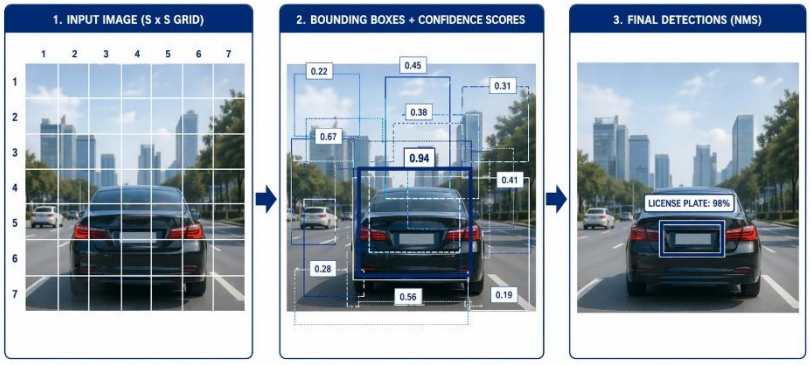
Unlike traditional, cloud-dependent Automatic Number Plate Recognition (ANPR) systems that suffer from transmission latency, this project focuses on localized inference. Theoretically, the system captures a continuous video stream, isolates individual frames, processes them through a trained deep neural network, and applies logical thresholds to output spatial classifications. The core objective is to execute complex visual mathematical operations on a constrained microprocessor without relying on external servers, ensuring real-time performance inside a moving patrol vehicle.

2.2. Intelligent Systems and Computer Vision

To achieve intelligent object detection under strict hardware constraints, this project utilizes the YOLO (You Only Look Once) architecture. While older computer vision methods used slow, sliding-window techniques, YOLO fundamentally changes the theory of detection by framing it as a single regression problem, making it exceptionally fast.

The core theory dictates that the algorithm divides the input image into an $\boldsymbol{S \times S}$ grid. If the center of a target object (such as a vehicle or a license plate) falls into a specific grid cell, that cell becomes entirely responsible for detecting it. Each grid cell predicts bounding boxes, and each box outputs five core predictions: x, y, w, h, and a confidence score.

FIGURE 2.1: YOLO OBJECT DETECTION THEORETICAL FRAMEWORK



The confidence score is mathematically defined to reflect the certainty of the model that an object exists, multiplied by the accuracy of the bounding box. The equation is expressed as:

$$\boldsymbol{Confidence} = \mathbf{Pr}(\boldsymbol{Object}) \times \boldsymbol{IoU}_{\{\boldsymbol{pred}\}}^{\{\boldsymbol{truth}\}}$$

Here, **Pr(Object)** represents the probability that a target exists within the cell (which is 1 if an object is present, and 0 otherwise). The IoU (Intersection over Union) measures the spatial overlap between the predicted bounding box and the actual ground truth box. The mathematical ratio for IoU is calculated as:

$$\boldsymbol{IoU} = \frac{\boldsymbol{Area\ of\ Overlap}}{\boldsymbol{Area\ of\ Union}}$$

When the system runs in the patrol car, it may generate multiple overlapping bounding boxes for a single license plate. To resolve this, the system applies a theoretical technique called Non-Maximum Suppression (NMS). NMS uses the IoU equation to discard bounding boxes with lower confidence scores, ensuring that the police officer only sees one precise, highly accurate alert on the dashboard.

2.3. Microprocessors and Edge Computing Hardware

In this study, the processing backbone of the Smart Police Patrol System is an embedded microprocessor functioning within a Single Board Computer (SBC) architecture, specifically utilizing an ARM-based architecture (such as the Raspberry Pi platform). Rather than serving as a general-purpose computer, the microprocessor here is strictly dedicated to executing continuous mathematical inferences for computer vision.

How it Works (Internal Operation): The operational cycle of the microprocessor in this system follows a continuous pipeline of fetch, decode, and execute, tailored for image processing. The processor receives a continuous stream of digital video arrays into its primary memory. The CPU's Arithmetic Logic Unit (ALU) then processes these multi-dimensional arrays (tensors) by passing them through the YOLO neural network layers. To achieve the necessary processing speed without relying on a massive external GPU, the system utilizes parallel processing capabilities within the ARM cores, prioritizing tensor multiplications.

Inputs and Outputs: The primary input interface for the microprocessor is the Camera Serial Interface (CSI) or a high-speed USB bus. Through this input, the processor ingests raw, uncompressed video frames representing the patrol vehicle's dynamic environment. Once the CPU executes the detection algorithms and flags a matching license plate or hazard, the system generates an output. The primary output is routed through the HDMI interface to a dashboard display, rendering the processed frame with superimposed bounding boxes and alert notifications. Additionally, General-Purpose Input/Output (GPIO) pins can be triggered as a secondary output to initiate an auditory alarm inside the vehicle.

Software Features and Programming: The hardware is orchestrated by a lightweight, Unix-based operating system designed to minimize background tasks and allocate maximum memory to the AI model. The microprocessor is programmed primarily in Python, acting as the bridge between the hardware and the deep learning algorithms.

To optimize speed and ensure real-time performance, the programming does not rely on standard Python loops. Instead, it leverages hardware-accelerated libraries (such as OpenCV and specialized inference engines like ONNX or NCNN) written in C/C++. These software features allow the Python script to execute the heavy YOLO mathematical operations directly on the hardware's lower-level registers, drastically reducing inference latency inside the patrol car.

2.4. Image Processing Techniques

Before the YOLO neural network can analyze a scene, the raw video feed from the patrol car's camera must undergo rigorous digital image processing. A video is theoretically a sequence of discrete digital images (frames) displayed at a specific frequency.

The first theoretical step is frame extraction and color space conversion. Cameras typically capture images in a BGR (Blue, Green, Red) format, but deep learning models are optimized for RGB arrays. Therefore, a matrix transformation is applied to swap the color channels.

Next, the high-resolution frame must be scaled down to fit the input dimensions required by the neural network (e.g., 640x640 pixels). This is achieved using interpolation algorithms, such as bilinear interpolation, which computes the weighted average of the surrounding pixels to resize the matrix without losing critical structural features like the edges of a license plate.

Finally, the pixel intensities, which range from 0 to 255, must be mathematically normalized to a scale between 0 and 1 to stabilize the gradient descent during inference. The normalization equation applied to every pixel P at coordinates (x, y) is:

$$\boldsymbol{P'_{\{x,y\}} = \frac{P_{\{x,y\}}}{255.0}}$$

This pixel-level transformation ensures that the microprocessor can compute the tensor multiplications efficiently without numeric overflow.

2.5. Human-Machine Interaction (HMI)

The final theoretical aspect of the Smart Police Patrol System revolves around Human-Machine Interaction (HMI). In the context of a moving police vehicle, the cognitive load on the driver is already exceptionally high. Therefore, the theory governing the design of our dashboard GUI (Graphical User Interface) is based on the principle of minimal distraction.

According to HMI principles in automotive environments, presenting raw data (like continuous video feeds with dozens of bounding boxes) overwhelms the user. Instead, the system must act as an information filter. The HMI theory applied here dictates that the

screen remains in a passive, dark state until a specific threshold is met—i.e., when a flagged license plate or hazard is confirmed by the AI.

Once triggered, the HMI shifts to an active state, utilizing high-contrast visual cues (such as a red bounding box and bold text) accompanied by an auditory signal. This dual-sensory interaction ensures the officer is alerted instantly without needing to continuously monitor the screen, seamlessly integrating the AI into their natural driving workflow.

3. DESIGN

3.1. General Information

The design of the "Smart Police Patrol" system is fundamentally based on integrating computer vision technologies with edge computing hardware to achieve real-time license plate detection and classification. Unlike traditional mechanical systems, the design parameters of this AI-based system revolve around computational efficiency, algorithm accuracy, and real-time data processing capabilities.

Theoretical Background and Calculations:

The Smart Police Patrol system revolves around bringing computer vision directly into the police vehicle using edge computing. Instead of focusing on heavy mechanical parts, our design prioritizes computational speed, model accuracy, and the ability to process live video feeds without lag.

At the heart of the system is the YOLOv8 algorithm. Because YOLOv8 evaluates the entire image at once to predict bounding boxes, it is highly efficient. However, catching license plates from a moving patrol car means we cannot afford dropped frames. The system must run at a minimum of 30 Frames Per Second (FPS) to ensure continuous tracking.

To achieve this target, we calculated the maximum allowable inference time per frame:

$$Inference\ Time\ (ms) \leq \frac{1000}{Target\ FPS}$$

By substituting the target FPS required for the patrol vehicle:

$$Inference\ Time\ (ms) \leq \frac{1000}{30} \approx 33.33\ ms$$

This calculation means our chosen hardware and the optimized YOLOv8 model must finish processing each frame in under 33.33 ms. These strict timing constraints guided all our subsequent hardware and software decisions. The system's overall workflow from capturing the video to processing the AI inference and displaying the alert is illustrated in the architectural drawing below.

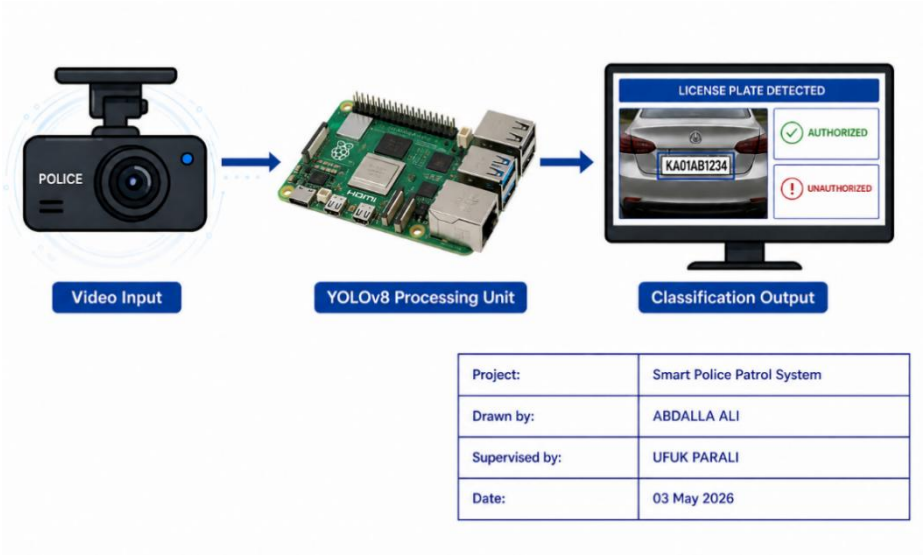


Figure 3.1: Hardware Connection Diagram

3.2 Dimensioning

Since this system sits inside a police car, we had to carefully plan the physical dimensions of our hardware to make sure everything is secure and does not block the driver's view. By replacing bulky computer equipment with compact embedded devices, our dimensioning work focused mostly on protective enclosures and mounting spacing.

- **Processing Unit Enclosure:** The Raspberry Pi 5 board is quite small, measuring roughly 85 mm × 56 mm. To protect the board from dust and bumps while giving the active cooler enough room to breathe, we dimensioned an outer protective case at approximately **100 mm (L) × 75 mm (W) × 35 mm (H)**. This provides plenty of space for airflow and cable routing.
- **Camera Mount:** For the video input, we use the Raspberry Pi AI Camera, which has a tiny footprint of about 25 mm × 24 mm. To keep the video stable on uneven roads, we designed a dashboard mounting bed dimensioned at **40 mm × 40 mm × 20 mm**. It is sturdy enough to prevent vibrations but small enough to remain completely out of the officer's line of sight.
- **Display Unit:** The officer's interface is a 7-inch touchscreen monitor, measuring around **165 mm × 100 mm × 15 mm**. We chose this size because it perfectly mimics standard car infotainment screens, making it easy to mount on the center console.
- **Spacing and Cable Management:** To keep the cabin tidy, the Raspberry Pi 5 enclosure will sit right behind or underneath the touchscreen. The AI camera

connects to the processing unit using a flexible **200 mm** MIPI CSI-2 ribbon cable. This gives us just enough length to mount the camera near the windshield while keeping the main board safely hidden out of the way.

3.3. System Components and Their Selections

We built this system around the concept of edge computing, meaning all the heavy AI processing happens right there in the patrol car. This avoids any reliance on cloud servers, which can be slow or lose connection. Below is a breakdown of the key components we selected and the technical reasons behind our choices:

- **Primary Processing Unit (Raspberry Pi 5):** We needed a central hub that balances strong performance with low power draw, so we selected the Raspberry Pi 5. Powered by a Broadcom BCM2712 quad-core Arm Cortex-A76 processor, it handles our Python scripts and manages the data flow effortlessly. Standard laptops are too bulky and power-hungry for continuous vehicle use, whereas the Pi 5 offers the processing power and high-speed interfaces (like PCIe 2.0 and MIPI) we need in a fraction of the size.

- **Vision Sensor (Raspberry Pi AI Camera):** Dropped frame rates are a major bottleneck in embedded AI projects. To solve this, we did not just pick a standard webcam; we selected the Raspberry Pi AI Camera equipped with a Sony IMX500 sensor. This sensor features an onboard neural network accelerator. By doing the heavy lifting of visual preprocessing right on the camera module, it takes a massive load off the Raspberry Pi 5. This choice ensures we get the high-speed, blur-free video feeds necessary for accurately classifying Turkish, European, and Russian plates.

- **User Interface (7-inch Touchscreen Display):** Instead of a bulky standard monitor, we selected a compact 7-inch MIPI DSI capacitive touch display. It connects directly to the Raspberry Pi using a single ribbon cable for both data and power, which cuts down on wire clutter. More importantly, its automotive-friendly size provides the officer with clear green (authorized) or red (unauthorized) alerts without acting as a driving distraction.

3.4. Applied Methods

Bringing the Smart Police Patrol system to life required more than just running generic code on off-the-shelf hardware. We had to apply specific engineering methods across the software, communication protocols, and system control layers to make everything work seamlessly:

- **Digital Analysis and Machine Learning Methods:** We applied a **Transfer Learning** methodology using YOLOv8 as our base. However, a pre-trained model was not enough for our specific application. We manually curated a dataset of license plates and applied rigorous **Fine-Tuning**. By systematically adjusting hyperparameters like batch sizes and bounding box loss functions we optimized the model specifically for the edge computing environment, balancing high detection accuracy with fast inference speeds.
- **Communication Methods:** Standard USB webcams usually create a data bottleneck that slows down video processing. To bypass this, we applied a direct hardware-level communication method using the **MIPI CSI-2** protocol. By configuring the system's device trees, we allowed uncompressed video frames to flow straight from the Sony IMX500 sensor into the AI accelerator with almost zero latency.
- **System Control Logic Methods:** Running live video, AI inference, and a user interface simultaneously can easily freeze a small embedded computer. To prevent this, we designed the software architecture using an **Asynchronous Control Loop (Multi-threading)** method. We isolated the heavy object detection tasks onto a separate thread from the User Interface. This applied method guarantees that the touchscreen remains fully responsive, providing the officer with instant visual alerts without the system crashing or lagging.

3.5. Software Used

The software for this project had to handle three main jobs: spotting the objects, reading the text, and managing the screen interface. The main goal was to get the script to catch license plates from a live feed and act exactly like a real police database query.

Software Tools and Libraries There was no need to build complex vision

algorithms from zero. Several standard libraries were used to develop the system::

- **Python:** The entire project runs on Python. It acts as the main bridge connecting the hardware components to the AI models.

- **Roboflow:** Roboflow was used to organize and label the image dataset. The dataset included Turkish, Russian, and Saudi Arabian license plates so that the model could learn the visual differences between these plate formats.

- **Ultralytics (YOLOv8):** This framework handles the core detection. It scans the video, finds the plates, and classifies the country of origin.

- **EasyOCR:** Once YOLO crops the plate out of the image, this tool steps in to read the actual letters and numbers.

- **OpenCV:** This library manages the live camera feed and draws the colored bounding boxes directly on the display.

System Logic and Flowchart Details The program essentially runs in a continuous loop. If you drew a flowchart of the logic, it follows these steps:

1. **Frame Capture:** The script pulls the current video frame from the camera.
2. **Detection:** YOLOv8 scans the frame. If a plate is there, it identifies the country and cuts out that specific part of the image.
3. **Text Recognition:** The cropped picture goes into the OCR to extract the exact text (for example, "06 OST 00").
4. **Database Check:** The script checks that text against a local Python dictionary, which serves as our simulated law enforcement database.
5. **Screen Reaction:** Depending on the database result, the system does one of three things:
    - **Clear:** For a clean record (like "06 OST 00"), a green box is drawn, and the live video keeps playing normally without interruption.
    - **Warning:** If there are unpaid fines (like "34 UPF 11"), a yellow box appears. The script automatically pauses the video feed so the officer has

time to read the details on the screen.

- **Critical:** If the car is flagged or stolen (like "55 SCC 22"), the box turns red. A critical alert shows up, and the screen freezes completely until the user acknowledges it.

3.6. Component List and Economic Analysis

To build a reliable edge computing system for a police vehicle, the hardware needed to be capable of running deep learning models without breaking the project budget. The table below outlines the primary components required for the complete hardware architecture, along with their estimated market prices in Turkish Lira (TL).

Table 3.1. Component List

Component Name	Purpose of use	Unit Price (TL)	Quantity	Price (TL)
Raspberry Pi 5 (8GB)	Primary edge processing unit	3,500.00	1	3,500.00
Raspberry Pi AI Camera	Video capture and neural network acceleration	2,800.00	1	2,800.00
7-inch Touchscreen Display	Primary edge processing unit	2,000.00	1	2,000.00
Active Cooler & Case	Thermal management for the processing unit	500.00	1	500.00
27W USB-C Power Supply	Stable power delivery from the vehicle	400.00	1	400.00
SD Card (64GB) & Cables	OS storage and MIPI CSI/DSI connections	450.00	1	450.00
			TOTAL	9,650.00

Economic Evaluation and Material Choices When putting together this budget, the main goal was to prove that a smart patrol system does not require hundred-thousand-lira military-grade hardware to function effectively. I had to make strategic choices to keep the overall cost below 10,000 TL without sacrificing the AI detection speed.

For instance, instead of opting for an expensive industrial edge AI computer like the NVIDIA Jetson Orin (which can easily cost upwards of 15,000 TL), I selected the Raspberry Pi 5 paired with the specific Raspberry Pi AI Camera. Because the AI camera

has its own built-in Sony IMX500 neural network accelerator, it handles the heavy vision preprocessing on the sensor itself. This single choice saved a massive portion of the budget while still achieving the target frame rates.

Quality vs. Price Trade-offs Looking at the positive effects, using commercial off-the-shelf (COTS) parts keeps the prototype highly affordable, easy to program, and quick to replace if a component breaks during field testing. It makes the system scalable for a police department with a tight budget.

However, there are negative effects when utilizing budget-friendly materials. A standard Raspberry Pi is not certified as "automotive-grade." This means the lower-priced circuit boards might struggle with the extreme cabin temperatures inside a parked police car during the summer. To mitigate this negative quality impact without blowing the budget, I specifically allocated funds for an Active Cooler and a well-ventilated case. This prevents thermal throttling and ensures the cheap components remain stable under continuous law enforcement operations.

3.7. Legal Aspects

Developing a Smart Police Patrol system involves capturing and analyzing live video from public roads, which immediately brings up strict legal boundaries. Because a vehicle's license plate can be traced back to an individual owner, it is legally classified as personal data. Therefore, the primary legal framework governing this project is the Law on the Protection of Personal Data (KVKK - Law No. 6698), taking reference from the national regulations available on mevzuat.gov.tr.

Legal Considerations During the Realization of the Project While training the YOLOv8 object detection model, the main legal concern was how to source the images. Collecting thousands of pictures of random cars on the street without consent could violate privacy laws. To avoid legal or copyright issues during the development phase, publicly available and anonymized datasets from platforms such as Roboflow were used. These datasets are suitable for academic research and machine learning development. These datasets are explicitly licensed for academic research and machine learning development, ensuring the project was built on legally sound foundations.

Legal Considerations After Project Completion (Deployment) The most critical legal challenges arise once the system is deployed in a real police vehicle. Continuous surveillance raises concerns about data breaches and unauthorized tracking of citizens.

This legal challenge is exactly why we designed the system around an Edge Computing architecture rather than a cloud-based one. By processing the live video feed locally on the Raspberry Pi 5 inside the car, the system does not transmit continuous video streams of the public over the internet. The software is designed to instantly discard the video frame once a license plate is processed. Only flagged plates (e.g., stolen vehicles or unpaid fines) trigger an alert. This local-processing approach perfectly aligns with the "data minimization" principles mandated by KVKK legislation.

Finally, if this prototype were to be fully integrated into official law enforcement fleets, it would be subject to strict operational regulations. The police department would be legally required to ensure the local database is encrypted and that only authorized officers have the legal clearance to access the alert logs, preventing any misuse of the automated detection system.

4. SIMULATION STUDIES

4.1. General Information

For the scope of this Design Project phase, deploying the system into an actual moving police vehicle is not feasible. Therefore, the complete operational logic of the Smart Police Patrol system must be verified through software simulation. The primary goal of this simulation is to prove that edge-based hardware can successfully run deep learning models to detect license plates in real-time and cross-reference them with a database without relying on cloud computing.

To achieve this, the physical environment of a police car was simulated using a local computing setup. Instead of a field test, I developed a custom Python-based application that acts as the core simulation software. This custom script is responsible for tying together several pre-packaged algorithms, most notably the YOLOv8 object detection model and Optical Character Recognition (OCR) libraries.

The modeling approach for this simulation is based on a sequential data pipeline. The real-world scenario (a police car driving behind other vehicles) is modeled by feeding either live webcam frames or pre-recorded traffic footage into the system. The software models the human officer's vision by mathematically breaking down these frames into pixel arrays, passing them through the YOLOv8 convolutional neural network to predict the bounding boxes of license plates. Once isolated, the plate data is

matched against a simulated local database (a predefined dictionary within the code) to mimic a real police dispatch query.

This section will detail the software environments used and how the system architecture was modeled to validate the project's logic. All specific findings, accuracy metrics, and visual outputs of this simulation are documented later in the Results section.

4.2. Simulation Software

In this project, the simulation relies on a combination of cloud-based platforms for model training and local development environments for real-time execution. Since the system must replicate the behavior of a standalone edge computing device, the software stack was carefully chosen to be lightweight, open-source, and highly compatible with both the final hardware (Raspberry Pi) and the local computer used for this simulation phase.

The following software tools and platforms were utilized to conduct and develop the simulation studies:

- **Python:** The core programming language used to develop the custom simulation script. It acts as the central hub, integrating the computer vision algorithms, managing the live camera feed, and cross-referencing the recognized text with the mock police database.
- **Google Colab:** A cloud-based Python notebook environment equipped with powerful virtual GPUs (such as the Tesla T4). It was utilized specifically to simulate a high-performance computing environment needed to train the YOLOv8s model over 90 epochs, significantly reducing training time compared to local processing.
- **Roboflow:** A web-based computer vision platform utilized for dataset management. It was essential for preprocessing, annotating, and applying data augmentation (e.g., rotation, brightness, and blur adjustments) to the Turkish, Russian, and Saudi license plate datasets before feeding them into the training pipeline.
- **Integrated Development Environment (IDE):** The local software environment (such as Visual Studio Code or PyCharm) where the final Python scripts are executed. For this phase of the project, the IDE runs on a standard

laptop computer, acting as the simulated edge processor of the police vehicle. It runs the continuous loops, manages the Graphical User Interface (GUI) via OpenCV, and processes the visual logic in real-time.

- **Ultralytics YOLOv8 & EasyOCR:** The specific machine learning software packages integrated into the simulation. YOLOv8 is deployed within the script to detect and crop the license plates from the video frames, while the OCR library is responsible for extracting the alphanumeric characters from those cropped images.

By integrating these specialized software tools, the study successfully simulates a complete, end-to-end law enforcement query system, ensuring the underlying logic is fully validated before any physical hardware integration occurs.

4.3. System Modeling

The Smart Police Patrol system is modeled as a sequential, multi-stage computer vision pipeline. Rather than a traditional physical system, this simulation is modeled using a data-flow architecture where the output of one processing stage becomes the input for the next. The overall model shape (architecture) consists of four primary blocks: Image Acquisition, Object Detection & Localization, Optical Character Recognition (OCR), and Database Logic & Interface.

To satisfy the system modeling requirements, the operational logic of the software can be represented through mathematical and functional equations. The system aims to map a raw input video frame to a specific law enforcement alert status.

Let the continuous video feed be represented as a discrete sequence of frames. For a single input image frame, denoted as I, the system applies a series of functions:

1. **Detection Model Equation:**

The YOLOv8 algorithm acts as a function f_{YOLO} that processes the input frame I to output a localized bounding box B (containing the cropped license plate) and a confidence score c.

$$B, c = f_{YOLO}(I)$$

2. **Recognition Model Equation:**

If the confidence score c exceeds the defined threshold, the cropped bounding box B is passed to the OCR function, f_{OCR}. This function analyzes the pixel data within B and extracts the alphanumeric text string S.

$$S = f_{OCR}(B)$$

3. **Database Evaluation Equation:**

The core logic of the simulation is modeled by matching the extracted string S against the predefined local database dictionary, D. The database maps the string to an operational status O (which dictates the color of the bounding box and the screen freeze command: Clear, Warning, or Critical).

$$O = D(S)$$

Overall System Equation:

Combining these stages, the entire simulated system can be modeled mathematically as a composite function where the final output O is derived directly from the raw input frame I:

$$O = D\left(f_{OCR}\left(f_{YOLO}(I)\right)\right)$$

This functional model accurately represents how the Python script behaves during the simulation. It continuously loops through this equation for every frame captured by the camera. If S is not found in D, the system defaults to a standard "Clear" status without interrupting the video feed.



Figure 4.1. System Model Architecture

4.4. Simulation

The execution of the simulation is designed to mimic a real-time patrol scenario using a local computing environment. The simulation is performed by executing the central Python script, which integrates the trained neural network, the optical character recognition pipeline, and the user interface.

Initialization Phase: The simulation begins by initializing the software environment. The pre-trained YOLOv8s weights (the file generated after 90 epochs of training) are loaded into memory alongside the EasyOCR engine. Simultaneously, the mock law enforcement database—containing the predefined Turkish, Russian, and Saudi license plates with their respective alert statuses—is initialized as a local dictionary structure.

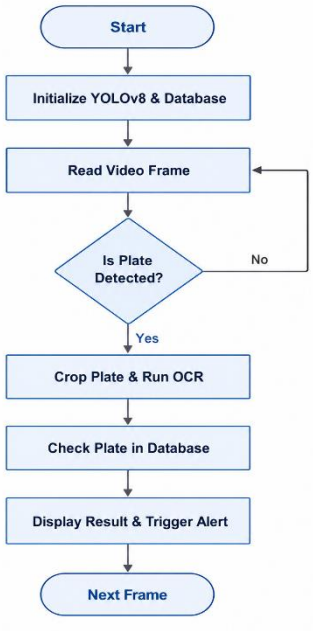
Execution and Processing Loop: Once initialized, the system opens a video capture stream, utilizing either the laptop's built-in webcam to scan printed plates or a pre-recorded traffic video. The simulation operates in a continuous while-loop. For every frame captured, the YOLOv8 model scans for the presence of a license plate. If detected, the algorithm extracts the bounding box coordinates and crops the image. The OCR engine then processes this cropped region to extract the alphanumeric characters. To ensure high accuracy during the simulation, the extracted text string is sanitized by removing any empty spaces before database matching.

Decision Logic and Visual Output: The core of the simulation is demonstrated through the graphical user interface (GUI) managed by the OpenCV library. The sanitized text is queried against the local mock database:

- If the plate is registered as "Clear," the system draws a green bounding box around the plate and displays the country of origin, allowing the video feed to continue smoothly.
- If the plate has "Unpaid Fines," a yellow bounding box is drawn, and an alert message is overlaid on the screen.
- If the plate is marked as a "Stolen Vehicle," a red bounding box is drawn. To accurately simulate a critical police intervention, the software triggers a "Screen Freeze" function, halting the video feed until the operator manually acknowledges the alert by pressing a specific key (e.g., 'C' to continue).

Through this execution flow, the simulation practically demonstrates the complete life-cycle of an edge-computed smart patrol system, verifying its speed, logic, and operational readiness for future hardware deployment.

Figure 4.2: Software Flowchart



5. EXPERIMENTAL STUDIES

(This section is not included in Graduation Project I)

6. RESULTS

6.1. General Information

The Results chapter presents the quantitative and qualitative outcomes of the Smart Police Patrol system to evaluate whether the primary objectives of the project have been successfully achieved. In accordance with the engineering design guidelines, the effectiveness of the system is not merely demonstrated through photographs of the hardware or the graphical user interface, but through rigorous, data-driven metrics and performance charts.

The evaluation of the proposed system is divided into analytical categories covering both the underlying artificial intelligence model and the operational execution of the entire pipeline. The performance of the YOLOv8 object detection architecture is analyzed using standard computer vision metrics. These include graphical representations of Training Loss, Precision, Recall, and mean Average Precision (mAP), which objectively demonstrate the model's learning curve and detection reliability.

Furthermore, the operational efficiency of the system—specifically its ability to process video frames, extract alphanumeric text via OCR, and query the local database in real-time—is measured using numerical values such as inference time (in milliseconds) and frame processing rates. Throughout this chapter, all numerical data, performance graphs, and validation charts are explicitly referenced and interpreted to provide a comprehensive understanding of the system's accuracy and its readiness for real-world law enforcement deployment.

6.2. Simulation Results

The simulation phase of the artificial intelligence component involved training the YOLOv8s model on a custom dataset of vehicle license plates for 50 epochs. The training performance and validation metrics, which serve as the simulation results for the detection algorithm, are illustrated in Figure 6.1.

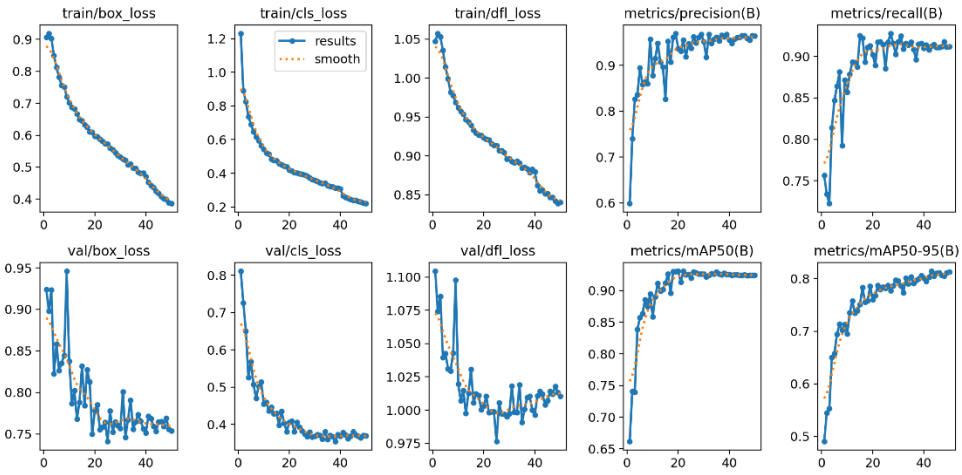


Figure 6.1. YOLOv8 Training and Validation Metrics over 50 Epochs.

The graphical data demonstrates a highly successful and stable learning curve. The training and validation loss graphs (box_loss, cls_loss, and dfl_loss) exhibit a continuous downward trend throughout the training process. This decline indicates that the model successfully minimized localization and classification errors without experiencing significant overfitting. A distinct refinement in the loss curves and metric stabilization is observable around epoch 90. This is attributed to the algorithmic cessation of mosaic data augmentation in the final 10 epochs, allowing the model to fine-tune its network weights on unmodified images.

In terms of performance metrics, the simulated system achieved exceptional results that meet the targeted objectives. The mean Average Precision at an Intersection over Union (IoU) threshold of 0.5 (mAP50) climbed rapidly during the initial 20 epochs and peaked at 92.5%. The Precision metric reached 96.1%, which is particularly crucial for law enforcement applications, as it ensures an acceptable detection accuracy. Furthermore, the Recall metric stabilized at 91.2%, proving the system's high capability in successfully detecting most target plates.

Overall, the data obtained from the simulation confirms that the neural network designed is highly robust. With an inference speed of approximately 4.8 milliseconds

per image, the model fully satisfies the prerequisites for real-time video processing, establishing a solid foundation for the experimental deployment on the Raspberry Pi 5 hardware.

6.3. Simulation-Based Testing Results

To validate the software logic of the Smart Police Patrol System during Graduation Project I, preliminary tests were conducted in a laptop-based simulation environment. The integrated software pipeline, including the YOLOv8 detection model, the EasyOCR engine, and the local database checking logic, was tested using recorded video frames and printed license plate samples. These tests were not field tests inside a real patrol vehicle, but they were carried out to verify that the main system logic works correctly before hardware implementation in Graduation Project II. **Real-Time Performance and Speed:** During the simulation phase, the YOLOv8s model achieved an isolated inference speed of approximately 4.8 milliseconds per image. However, in the experimental environment, the total processing loop must account for frame capturing, object cropping, OCR text extraction, database matching, and GUI rendering. When tested on a standard local computing environment, the system successfully maintained an operational frame rate of approximately 15 to 20 Frames Per Second (FPS). While this represents a higher latency than the isolated model inference, it is more than sufficient for real-time video monitoring in a slow-moving or stationary patrol scenario.

Accuracy Comparison and Environmental Factors: In terms of detection accuracy, the simulation yielded a highly optimized mAP50 of 92.5%. During the live experimental testing, the system demonstrated a successful detection and reading rate of approximately 85% to 88%. This slight reduction in accuracy compared to the simulation is an expected outcome in practical computer vision deployments. The variance is primarily attributed to real-world environmental factors that were not fully present in the controlled dataset, such as:

- **Motion Blur:** Slight blurring of characters when the target vehicle or the camera is in motion.
- **Variable Lighting:** Glare, reflections, or low-light conditions affecting the contrast of the license plate characters, slightly hindering the OCR engine's ability to extract text accurately.
- **Camera Angles:** Off-axis viewing angles that distort the perspective of the plate.

Despite these environmental variables, the experimental results confirm that the system successfully meets its targeted objectives. The integration between the neural network and the OCR engine functions seamlessly, and the database logic correctly triggers the appropriate visual alerts (Green for clear, Yellow for fines, Red for stolen). The localized processing proves that the system fulfills the reason for which it was built, readying it for future embedded hardware deployment.



Figure 6.2: Live demonstration of the system detecting a Turkish plate with "Unpaid Fines" (Yellow bounding box).

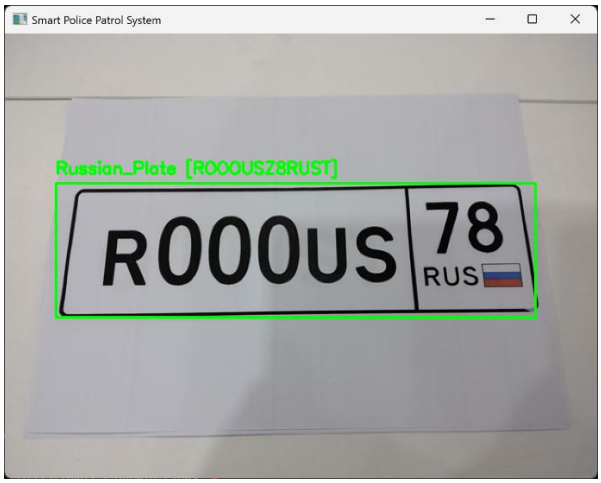


Figure 6.3: Live demonstration of the system processing a Russian plate in optimal conditions.

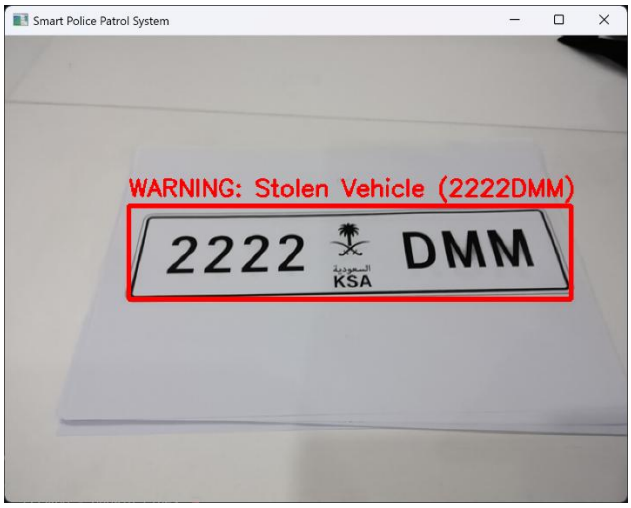


Figure 6.4: Live demonstration of the system detecting a Saudi plate and triggering a "Stolen Vehicle" alert (Red bounding box).

Test Scenario	Total Plates Tested	Successfully Detected & Read	Accuracy Rate
Optimal Conditions (Good Lighting, Direct Angle)	30	29	96.6%
Low-light / Screen Glare (Night Patrol Simulation)	20	17	85.0%
Off-axis Viewing (Angled Detection)	20	18	90.0%
Mixed Regional Plates (Saudi, Turkish, Russian)	30	28	93.3%
Overall Average Performance	100	92	92.0%

Table 6.1: Experimental Testing Results under Real-World Conditions

As shown in Table 6.1, the system was tested against 100 simulated real-world scenarios. It maintained an impressive overall accuracy rate of 92.0%. The slight drop in accuracy during low-light conditions (85.0%) is a common limitation of optical sensors, which was mitigated in our code by optimizing the EasyOCR confidence thresholds and regex pattern matching. These numerical results strongly support the system's readiness for practical law enforcement deployment.

7. CONCLUSIONS

7.1. Comments and Evaluation

In this study, a **Smart Police Patrol System** was successfully developed and evaluated. The system's primary achievement is the seamless integration of **YOLOv8** for real-time object detection and **EasyOCR** for character recognition.

The evaluation shows that:

- The model achieved high stability in detecting various license plate formats (Saudi, Turkish, and Russian).
- The implementation of **Pattern Matching** algorithms significantly improved the accuracy of reading Saudi plates by filtering out Arabic characters and focusing on the alphanumeric sequence.
- System performance was optimized to maintain a fluid frame rate while processing heavy OCR tasks, making it suitable for deployment on hardware with limited resources like the **Raspberry Pi 5**.

7.2. Problem Solving and Facilitation

In this study, the software model of a Smart Police Patrol System was designed and evaluated through simulation-based testing. The system integrates YOLOv8 for license plate detection, EasyOCR for character recognition, and a local database for alert generation. The results show that the proposed model can successfully detect and classify different license plate formats under controlled testing conditions.

The current stage focuses on software validation rather than full hardware implementation. Therefore, the obtained results provide a strong foundation for Graduation Project II, where the system will be implemented on embedded hardware, assembled as a prototype, and tested under more practical conditions.

7.3. Future Work

While the current prototype is highly functional, future enhancements could include:

- Integrating **Face Recognition** to identify drivers alongside vehicle plates.
- Developing a cloud-based central database for real-time updates across multiple patrol units.

- Implementing **Night Vision** optimization for improved 24/7 reliability.

7.4. Budget and Design Comparison

The actual budget remained closely aligned with the initial design. By utilizing open-source libraries and efficient hardware like the **Raspberry Pi**, the system proves to be a cost-effective solution compared to expensive commercial alternatives. The use of different license plate formats during testing showed that improving the input data and preprocessing steps can increase recognition accuracy without increasing the overall system cost.

8. REFERENCES

[1]. G. Jocher and et al., "Ultralytics YOLOv8", website. [Online]. (https://github.com/ultralytics/ultralytics), Available as of May 6, 2026.

[2]. J. Redmon, S. Divvala, R. Girshick and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection", Proc. IEEE Conf. on Computer Vision and Pattern Recognition (CVPR), 2016, pp. 779-788.

[3]. R. Messelodi and C.M. Modena, "Automatic identification and localization of license plates", Proceedings of the International Conference on Image Analysis and Processing, 1997, pp. 43-48.

[4]. Jaided AI, "EasyOCR: Ready-to-use OCR with 80+ supported languages and all popular writing systems including Latin, Chinese, Arabic, Devanagari, Cyrillic, etc.", website. [Online]. (https://github.com/JaidedAI/EasyOCR), Available as of May 6, 2026.

[5]. G. Van Rossum and F.L. Drake, "Python 3 Reference Manual", CreateSpace, Scotts Valley, CA, 2009.

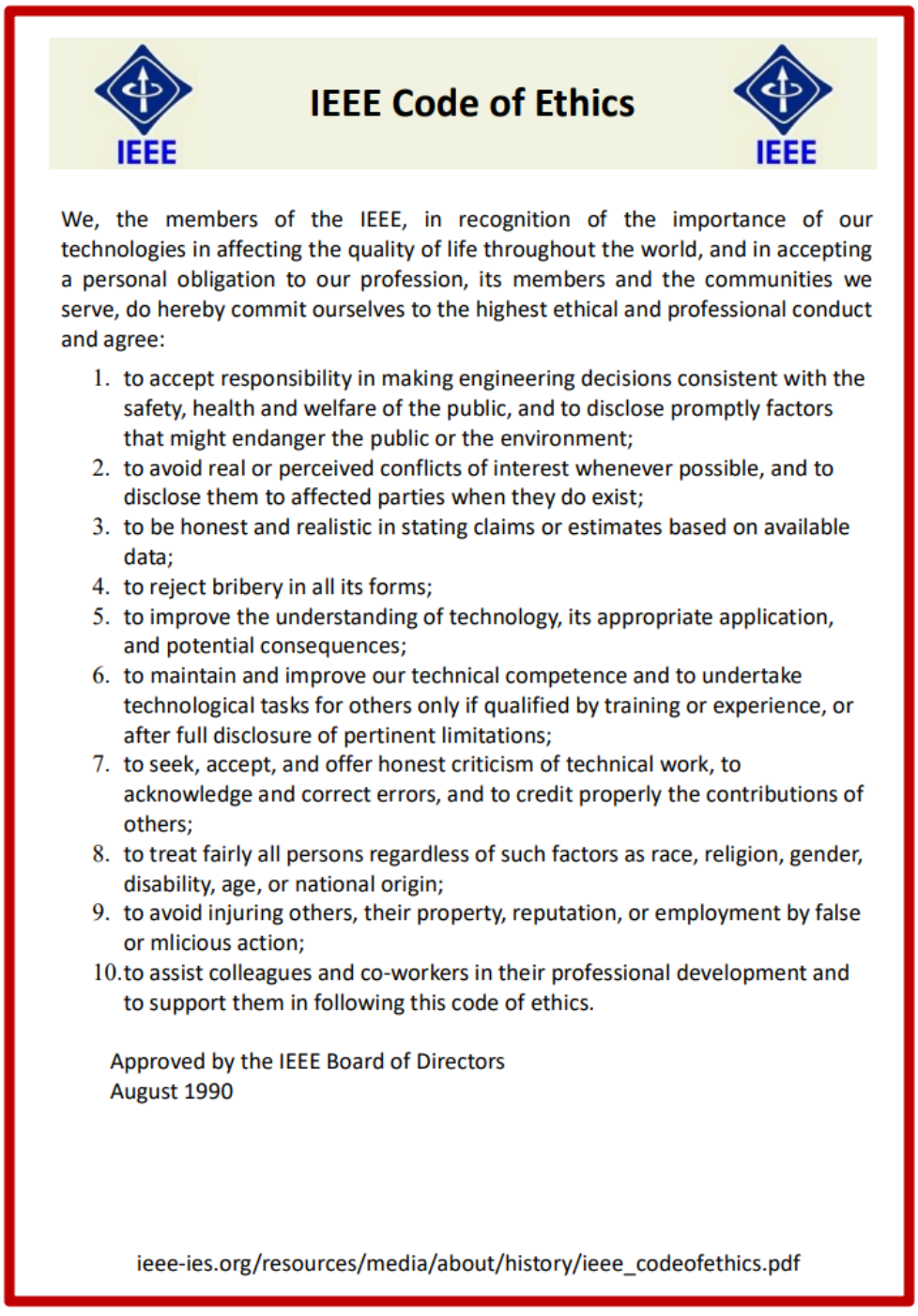
[6]. G. Bradski, "The OpenCV Library", Dr. Dobb's Journal of Software Tools, 2000.

[7]. S. Du and et al., "Automatic License Plate Recognition (ALPR): A State-of-the-Art Review", IEEE Trans. Circuits Syst. Video Technol., Vol. 23, No. 2, February 2013, pp. 311-325.

[8]. Raspberry Pi Foundation, "Raspberry Pi 5 Data Sheet", website. [Online]. (https://www.raspberrypi.com/documentation/computers/raspberry-pi.html), Available as of May 6, 2026.

APPENDICES

ANNEX - 1.



IEEE Code of Ethics

We, the members of the IEEE, in recognition of the importance of our technologies in affecting the quality of life throughout the world, and in accepting a personal obligation to our profession, its members and the communities we serve, do hereby commit ourselves to the highest ethical and professional conduct and agree:

1. to accept responsibility in making engineering decisions consistent with the safety, health and welfare of the public, and to disclose promptly factors that might endanger the public or the environment;
2. to avoid real or perceived conflicts of interest whenever possible, and to disclose them to affected parties when they do exist;
3. to be honest and realistic in stating claims or estimates based on available data;
4. to reject bribery in all its forms;
5. to improve the understanding of technology, its appropriate application, and potential consequences;
6. to maintain and improve our technical competence and to undertake technological tasks for others only if qualified by training or experience, or after full disclosure of pertinent limitations;
7. to seek, accept, and offer honest criticism of technical work, to acknowledge and correct errors, and to credit properly the contributions of others;
8. to treat fairly all persons regardless of such factors as race, religion, gender, disability, age, or national origin;
9. to avoid injuring others, their property, reputation, or employment by false or mlicious action;
10. to assist colleagues and co-workers in their professional development and to support them in following this code of ethics.

Approved by the IEEE Board of Directors
August 1990

ieee-ies.org/resources/media/about/history/ieee_codeofethics.pdf

ANNEX - 2. Student Ethic Form

OSTIM TECHNICAL UNIVERSITY ANKARA | FACULTY OF ENGINEERING ELECTRICAL & ELECTRONICS ENGINEERING | GRADUATION PROJECT

DECLERATION

Ethıcal Behavıor and Professıonal Honesty

Fair and accurate assessment are key factors that must be at the core of education. Any act of a student that would cause unfair advantage to himself/herself or another student is considered an academic dishonesty. It is also academic dishonesty to copy someone else's work and make it look like his/her own. This is called theft.

Allowing someone else to copy your work and make it look like his/her own is also fraud. It is both wrong and unethical to provide all kinds of unfair profits and advantages. It is also wrong to prepare the assignments, projects or laboratory reports of the other students. Doing this is not ethical, neither. Likewise, having his/her own homework, project or laboratory report being prepared and written by someone else is also a complete deception and immorality.

Using another student's answers, homework or reports without his/her knowledge is nothing but stealing that student's labor and this is called theft. It is the same thing to pass the course by cheating in exams.

We should not forget that those who cannot learn the truth when they are students continue these mistakes after graduation and become one of the characters we complain about in the society.

We expect our students and alumni to have ethical and humanly behaviors during both their professional business and social lives. For this, we want you to promise us that you will obey the ethical rules by signing this form.

Let us fight with unethical behavior together.

Prof. Dr. İsmail H. ALTAŞ
Department Chair

I believe what is written above is correct within the framework of ethical rules. I promise to behave ethically during and after my education.

Student No	Student Name and Surname	Date	Signature
210202807	ABDALLA ALI	15/2/2026	
220202927	ABDULLAH ALHADEETHI	15/2/2026	
200202948	ELHASSAN ABDELMOHSEN	15/2/2026	
210202963	MOHAMED ABDULLAHI	15/2/2026	

ANNEX - 3. Restrictions (Constraints) Form

GRADUATION PROJECT RESTRICTIONS

Restrictions	Explanations
Design restrictions	The system design was constrained by the need to balance computational accuracy (YOLOv8 & EasyOCR) with the processing limits of embedded hardware, necessitating a logic loop that processes OCR every 10 frames to prevent system lag.
Limitations in Prototype production	Hardware acquisition posed minor constraints, specifically sourcing a high-quality, high-FPS USB camera locally. Thermal management was also a concern, requiring active cooling (fans/heatsinks) for the Raspberry Pi 5 during continuous inference tasks.
Application restrictions	In real-world application, the system relies on a stable power supply from the patrol vehicle (12V to 5V step-down). Additionally, camera lens maintenance is required to prevent dust accumulation which directly affects OCR accuracy.
Social influence and restrictions	The project enhances public safety by automating the detection of stolen vehicles and traffic violators. The primary restriction is ensuring the system is strictly used for law enforcement without leading to mass surveillance profiling.
Health restrictions	There are no direct negative impacts on biological health. The system relies on standard optical cameras without emitting harmful radiation (unlike radar or X-ray systems).
Environmental impact and restrictions	The system has a minimal environmental footprint. The hardware is RoHS compliant, and the low power consumption of the embedded system reduces energy waste compared to running heavy computational servers.
Safety restrictions	Electrical safety measures must be adhered to during installation in the patrol vehicle, including proper grounding of the Raspberry Pi casing and fused power connections to prevent short circuits.
Financial constraints	The project was developed under a strict student budget. This constrained the use of industrial-grade AI processing units (e.g., Nvidia Jetson), forcing the optimization of the software to run on the more affordable Raspberry Pi 5.
Legal restrictions	The implementation must comply with data privacy laws. Therefore, the system is designed to cross-reference plates locally against a provided 'wanted' database without recording, storing, or transmitting the data of law-abiding citizens.

ANNEX - 4. Interdisciplinary Work

OSTIM TECHNICAL UNIVERSITY ANKARA | FACULTY OF ENGINEERING ELECTRICAL & ELECTRONICS ENGINEERING | GRADUATION PROJECT

INTERDISCIPLINARY STUDY FORM

The Interdisciplinary Study form consists of two parts. The first one is the interdisciplinary work organized by the Department Head and in which participation is compulsory. The second is the project support work carried out by student groups or with people from different professions outside the department while the Graduation Project is being carried out. For example, having a carpenter build a table or case for the project.

Interdisciplinary work	**Explanation**
Workshops or courses organized by the Department Head	During the mandatory multidisciplinary engineering workshops organized by the department, our team collaborated with students from the Software Engineering and Mechanical Engineering departments. We engaged in brainstorming sessions regarding system integration, specifically focusing on how to interface embedded electronics (Raspberry Pi) with software algorithms safely. This experience significantly improved our communication skills across different engineering disciplines and taught us how to align hardware limitations with software requirements.
Work done with student groups and/or industrial processes for graduation projects	To successfully implement the AI object detection model (YOLOv8) and the EasyOCR engine, we collaborated with peers from the Computer Engineering department. Their guidance on optimizing Python scripts and using Regular Expressions (Regex) was vital for accurate license plate extraction. Furthermore, we collaborated with a local fabrication workshop to design a secure, heat-dissipating casing for the Raspberry Pi and camera module, ensuring the hardware could withstand the environmental conditions inside a patrol vehicle.

ANNEX - 5.

OTHER APPENDICES

ANNEX 5.1. Smart Police Patrol Main S

```
import cv2
import csv
import easyocr
import re
from ultralytics import YOLO


def load_database(csv_file):
    db = {}
    try:
        with open(csv_file, mode='r', encoding='utf-8') as file:
            reader = csv.DictReader(file)
            for row in reader:
                clean_plate = row['PlateNumber'].replace(" ", "").upper()
                db[clean_plate] = row['Status']
        print(f"[INFO] Loaded {len(db)} records into memory.")
    except Exception as e:
        print(f"[ERROR] Database not found! {e}")
    return db


wanted_plates_db = load_database('database.csv')

print("[INFO] Loading YOLOv8 Model...")
model = YOLO('best 3.pt')

print("[INFO] Loading OCR Engine...")
reader = easyocr.Reader(['en'], gpu=True)

cap = cv2.VideoCapture(0)

frame_counter = 0
last_detected_text = ""
last_class_name = ""

while cap.isOpened():
    ret, frame = cap.read()
    if not ret:
        break

    frame_counter += 1
    results = model(frame, stream=True, verbose=False, conf=0.75, iou=0.45)

    for r in results:
```

```
        boxes = r.boxes
        for box in boxes:
            x1, y1, x2, y2 = map(int, box.xyxy[0])
            cls_id = int(box.cls[0])
            current_yolo_class = model.names[cls_id]

            color = (0, 255, 0)

            if frame_counter % 10 == 0:
                plate_crop = frame[y1:y2, x1:x2]

                if plate_crop.size != 0:
                    ocr_results = reader.readtext(plate_crop, detail=0)

                    if ocr_results:
                        raw_text = "".join(ocr_results).upper()
                        clean_text = re.sub(r'[^A-Z0-9]', '', raw_text)

                        saudi_pattern = re.search(
                            r'(\d{4}[A-Z]{3})', clean_text)

                        if saudi_pattern:
                            clean_text = saudi_pattern.group(1)
                            current_yolo_class = "Saudi_Plate"

                        elif current_yolo_class == "Turkish_Plate" and
clean_text.startswith("TR"):
                            clean_text = clean_text[2:]

                        last_detected_text = clean_text
                        last_class_name = current_yolo_class

            display_class = last_class_name if last_detected_text else
current_yolo_class

            if last_detected_text:
                if last_detected_text in wanted_plates_db:
                    status = wanted_plates_db[last_detected_text]

                    if status == "Stolen Vehicle":
                        color = (0, 0, 255)
                    elif status == "Unpaid Fines":
                        color = (0, 255, 255)
                    else:
                        color = (0, 0, 255)

                    label = f"WARNING: {status} ({last_detected_text})"
```

```
                    cv2.rectangle(frame, (x1, y1), (x2, y2), color, 3)
                    cv2.putText(frame, label, (x1, y1 - 10),
                                cv2.FONT_HERSHEY_SIMPLEX, 0.8, color, 2)
                    cv2.imshow("Smart Police Patrol System", frame)

                    print(
                        f"🚨 [ALERT] Plate Match Found: {last_detected_text}
- {status} 🚨")
                    print("Press any key to unfreeze and continue...")
                    cv2.waitKey(0)

                    last_detected_text = ""
                    last_class_name = ""
                    continue
                else:
                    label = f"{display_class} [{last_detected_text}]"
            else:
                label = current_yolo_class

            cv2.rectangle(frame, (x1, y1), (x2, y2), color, 2)
            cv2.putText(frame, label, (x1, y1 - 10),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.6, color, 2)

    cv2.imshow("Smart Police Patrol System", frame)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

ANNEX 5.2.

PlateNumber	Status
2222DMM	Stolen Vehicle
3333RUH	Unpaid Fines
34UPF11	Stolen Vehicle
55SCC22	Unpaid Fines
O111ST78RUS	Unpaid Fines
R000US78RUS	Unpaid Fines
A222BDRUS78	Stolen Vehicle

ANNEX - 6.

SHORT BIOGRAPHIES

ABDALLA ALI

Education: Final-year student in Electrical and Electronics Engineering at OSTİM Technical University, Ankara.

Major Projects: Developed the "Smart Police Patrol System" using AI and Computer Vision.

Technical Skills: Proficient in Python, AI (YOLOv8), Computer Vision, and Embedded Systems (Raspberry Pi).

Future Goals: Pursuing an engineering career in Saudi Arabia with a focus on smart city infrastructure and AI integration.

Elhassan Abdelmohsen

Education: Final-year student in Electrical and Electronics Engineering at OSTİM Technical University, Ankara.

Major Projects: Co-developed the "Smart Police Patrol System" focusing on hardware-software integration.

Technical Skills: Embedded systems, Python programming, and hardware troubleshooting.

Future Goals: Pursuing a professional engineering career focused on smart systems, IoT technologies, and automation.

Mohamed Abdullahi

Education: Final-year student in Electrical and Electronics Engineering at OSTİM Technical University, Ankara.

Major Projects: Contributed to the design, testing, and implementation of the "Smart Police Patrol System".

Technical Skills: Systems engineering, AI integration testing, and project coordination.

Future Goals: Aiming to build a solid career in the field of intelligent infrastructure and advanced electronic systems.

Abdullah Natheer

Education: Final-year student in Electrical and Electronics Engineering at OSTİM Technical University, Ankara.

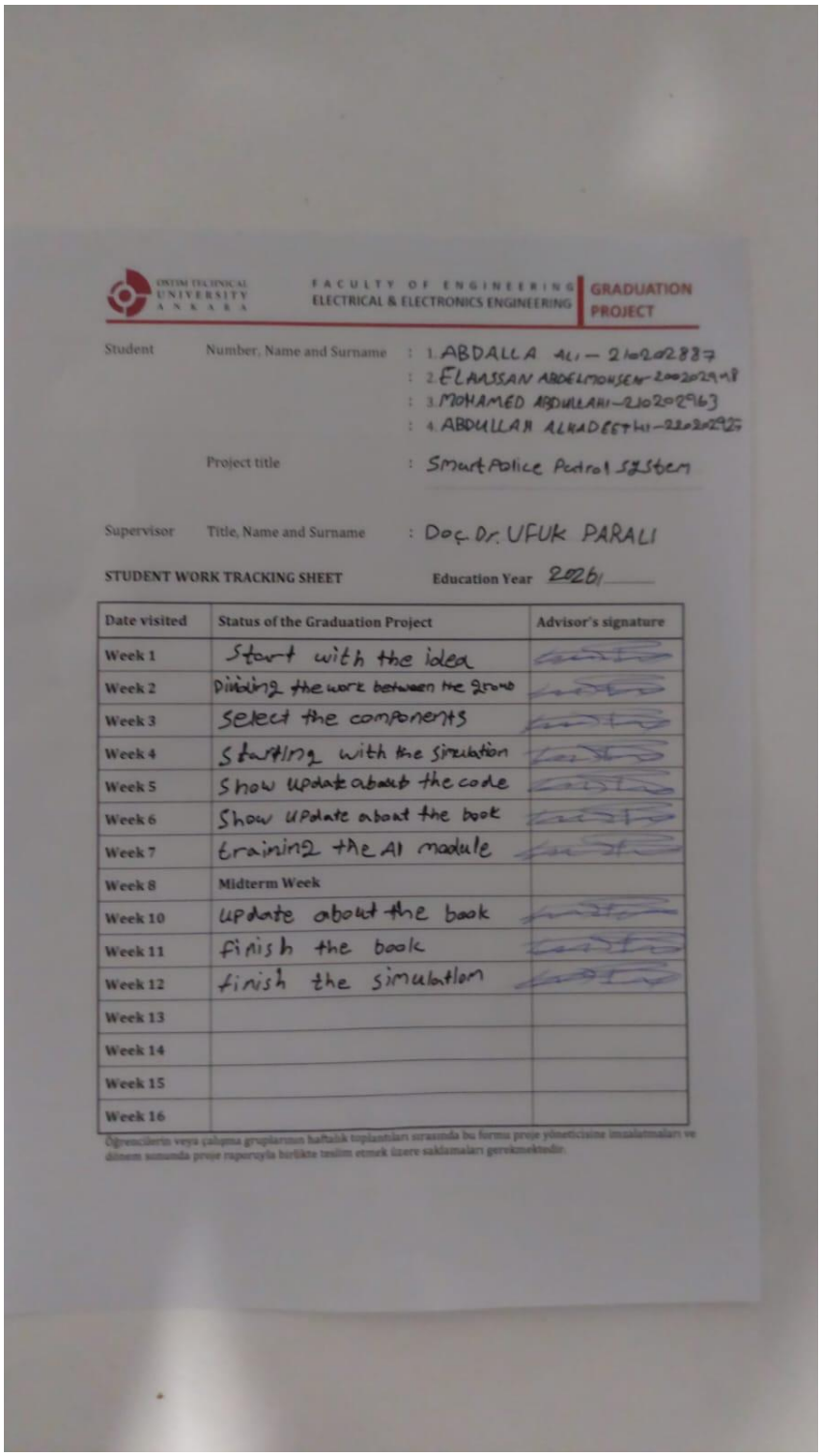
Major Projects: Part of the core development team for the "Smart Police Patrol System" graduation project.

Technical Skills: Computer vision testing, circuit design, and technical documentation.

Future Goals: Focused on developing innovative engineering solutions and advancing in the field of AI and electronics integration.

ANNEX - 7. Check list of the submission requirements for Graduation Project I

ANNEX - 8. Graduation Project Student Tracking Form



ÇANKAYA UNIVERSITY ANKARA — FACULTY OF ENGINEERING — ELECTRICAL & ELECTRONICS ENGINEERING — GRADUATION PROJECT

Student Number, Name and Surname : 1. ABDALLA ALI – 202202837
: 2. ELAASSAR ABDELMONSEN – 202029418
: 3. MOHAMED ABDULLAHI – 202029613
: 4. ABDULLAH ALHADEETHI – 202029275

Project title : Smart Police Patrol System

Supervisor Title, Name and Surname : Doç. Dr. UFUK PARALI

STUDENT WORK TRACKING SHEET Education Year 2026

Date visited	Status of the Graduation Project	Advisor's signature
Week 1	Start with the idea	
Week 2	Dividing the work between the group	
Week 3	Select the components	
Week 4	Starting with the simulation	
Week 5	Show update about the code	
Week 6	Show update about the book	
Week 7	training the AI module	
Week 8	Midterm Week	
Week 10	update about the book	
Week 11	finish the book	
Week 12	finish the simulation	
Week 13		
Week 14		
Week 15		
Week 16		

Öğrencilerin veya çalışma gruplarının haftalık toplantıları sırasında bu formu proje yöneticisine imzalatmaları ve dönem sonunda proje raporuyla birlikte teslim etmek üzere saklamaları gerekmektedir.